

0. 법적 문제 방지를 위한 info 추가

SPRED는 연습용 악보 제작 및 재생 보조 프로그램이며, 음원 파일의 업로드, 다운로드, 저장, 캐싱, 추출 기능을 제공하지 않는다.

외부 영상은 공식 **embed player**를 통해서만 재생되며, 프로그램은 영상 또는 음원의 복제물을 생성하지 않는다.

공식 예제 악보는 직접 제작한 음악, 퍼블릭 도메인 음악, 또는 악보 변환과 재배포가 허용된 공개 라이선스 음악으로 제한한다.

사용자가 작성한 악보와 메타데이터에 대한 권리 확보 책임은 사용자에게 있으며, 저작권자의 허락 없이 보호되는 음악, 가사, 악보, 음원 데이터를 배포해서는 안 된다.

SPRED는 가사 데이터, 저작권 있는 음악 라이브러리, 음원 파일 공유 기능, 사용자 간 악보 공유 플랫폼을 제공하지 않는다.

저작권 관련 문의 또는 삭제 요청은 지정된 이메일로 접수할 수 있으며, 권리 침해 소지가 확인된 공식 예제 또는 배포 자료는 수정·삭제될 수 있다.

1. UI

노래방 모드는 기존 분석 결과를 수정하지 않고, 재생 중인 분석 결과를 참조하는 실시간 평가 계층이다.

플레이어 영역 우하단에 “game mode”라고 적힌 버튼을 둔다.

마이크가 없으면 버튼을 비활성화한다. 마이크가 있다면 버튼 클릭시 **AppState**를 “game”으로 전환한다.

이때 **broadcast window**을 띄운다. 위치 후보는 **player** 영역이며, 그 위의 빈 영역에 버튼과 글자를 덧씌운 형태를 생각중이다. 투명한 **broadcast layer**가 필요할 수 있다.

표기해야 하는 정보와 버튼은 다음과 같다.

- 기존의 **player** 영역에 존재하던 재생 버튼, 정지 버튼, **seek bar**, 오실레이터 볼륨

- bar**, 작곡가와 곡 제목 정보, 현재 **BPM**

- 점수 관련 정보 (정확도, 처리한 이벤트 수, 총점)

- **broadcast window**와 “game” 상태를 종료하는 “exit practice mode” 버튼

- 플레이 안내 문구 (판정 기준, 엄밀하지 않음, 잡음이 없는 환경 및 마이크와 헤드셋 사용 권장 등의 정보)

“game” 상태에서 실행 가능한 요소와 실행 불가능한 요소는 다음과 같다.

실행 가능 : 유튜브 동시 재생, 처음부터 재생, 일시정지, 처음으로 돌아가기

실행 불가능 : 에дит 모드, 점프, 루프. 그 이외의 모든 버튼 조작도 우선 **disabled**로 간주한다.

2. 마이크 입력의 피치 인식과 시각화

악보 영역의 가장 왼쪽 재생 기준선 인근 영역에 작은 형광초록색 원으로 현재 인식된 음정을 표시한다. 레이아웃 범위를 벗어날 경우 원이 최상단 또는 최하단 행에 고정된다.

잡음 제거 기능은 관련 라이브러리가 존재하는지 탐색한 후 있다면 도입한다.

3. 판정과 점수 산정

점수는 우선 세 종류 존재한다.

- 전체 평균 음정 정확도 : 처리한 모든 이벤트의 음정 정확도의 평균
- 이벤트 처리 개수 : 처리한 **NoteEvent**의 총 개수
- 총점 : 처리 이벤트와 각 이벤트의 정확도를 같이 산정한 점수

우선 잡음을 제거하고, 선명하게 들리는 연주 소리만 인식(라이브러리 사용 권장)하여 현재 연주 피치 (**UserPitch**)로 본다.

현재 재생중인 **active track**의 **NoteEvent**들 중 음정이 가장 가까운 것을 중심으로 판정한다. 또는 앞서 구현한 음정 시각 효과와 노트 사각형의 좌표가 겹치는 정도를 측정하여 정확도를 산정할 수도 있겠다. 이는 더 비용이 적은 쪽을 선택한다.

생각중인 음정 정확도 판정은 다음과 같으며, 이후 다른 리듬게임이나 판정 프로그램을 참고하여 조정할 수도 있다.

기본 **NoteEvent**의 정확도 산정 (옥타브가 다르더라도 동일 계이름이라면 정답으로 인식하면 좋음)

오차범위	정확도	표시 문자
50cent 미만	100%	Good
100cent 미만 50cent 이상	80%	Ok
200cent 미만 100 cent 이상	20%	Hmm
200cent 이상	0%	Nope

표시 문자의 경우 리듬게임의 노트 처리시 나오는 효과를 생각하면 된다. 복잡한 **CSS**를 넣지는 않고, 그냥 재생 기준선과 노트 이벤트가 만나는 지점 근처에 색깔이 서로 다른 문자열을 출력하는 수준으로 한다.

일정 시간(ex: $0.5 \text{ initialTick} + n \text{ currentTick}$)마다 **UserPitch**와 현재 재생중인 **NoteEvent**를 비교하여, 정확도의 평균을 매긴다.

총점은 다음과 같이 매길 수 있다. 이는 확정 사항은 아니며 후보 중 하나이다.

```
... 의사코드
MiddleGameScore = 0;
for(정확도 산정할 때마다) {
    MiddleGameScore += 처리한 NoteEvent의 Track에 부여된 난이도 * 정확도
}
TotalGameScore = MiddleGameScore * 전체 평균 정확도
...
```

정확도는 의도적으로 두번 반영한다.

추가적인 판정 후보 (점수 증폭률은 정확도와 별개의 수치며, 정확도 판정 기준은 기본적으로 기존과 동일하다)

이벤트	점수 증폭률	표시 문자
gliss	100%	slide!
vib	120%	vibe!
trem	150%	trem!

비브라토의 경우 의도적으로 음을 떨어야 하므로 해당 구간은 음 떨림을 감지하면 오히려 가산점을 주는 것이 타당하다. 하지만 어떻게 “틀린 음”과 “비브라토”를 구분할지는 추가 논의가 필요하다. 비브라토 구간의 시작부터 끝까지 음정의 평균을 매겨서 **Good** 범위에 들어가는지 && 충분한 음 떨림이 감지되었는지를 조건으로 두고 비브라토가 끝나는 지점에서 **Vib!** 판정을 결정짓는 방안을 후보로 둘 수 있다.

글리산도의 경우 **anchor**의 노트 이벤트만 제대로 음정을 맞췄는지 확인하고, 그 사이에서는 음정이 연속적으로 끊김 없이 이어지기만 한다면 옳게 연주한 것으로 판정한다.

trem의 경우는 현재로서는 잘 모르겠다. 잘게 떠는 연주에서 아주 찰나에 일어나는 끊김 또는 볼륨 변화를 음정 인식 기능이 인식 가능할지 확실하지 않으므로, 현재는 구체적인 방법을 생각하지 못하고 구현 가능성만 열어둔 상태이다. 너무 까다롭다면 구현하지 않을 수도 있다.

추가적으로 음정 연주의 시작 시점을 정확하게 잡아낼 수 있다면 아래 점수 요소의 도입도 검토할 수 있다.

- 전체 평균 타이밍 정확도 : 처리한 모든 이벤트의 타이밍 정확도의 평균

판정이 앞선 점수 계산 공식에 적절히 추가한다.

4. 상태 천이

“game” 상태는 “edit” 상태, “loop” 등과 공존할 수 없다. 제한적인 동작만 지원한다.

재생 기능의 경우 일반 리듬게임과 동일하게 다음 두 가지만 지원한다.

1. 정지(처음으로 돌아가기) : 점수 리셋
2. 일시정지 및 다시 재생 : 점수 고정, 다시 재생 시 산정을 재개

재생상태 1에서는 시작 전 가벼운 설정이 가능하다. **volume bar**와 추가 예정 기능인 **speed bar**(colWidth를 변화시키는 기능), 유튜브 모드의 **on/off**와 **offset** 조절 기능을 여전히 설정 가능하다.

재생상태2에서는 그러한 조작이 불가능하며, 재생 상태 1에서 설정한 환경 설정 정보가 고정된다.

종료 상태가 존재하며, 천이 시점은 악보에 더 이상 재생할 **NoteEvent**가 없는 시점을 기준으로 한다. 이때 마지막으로 총점을 산정한다.

5. 구현 순서

1차 : 상태 추가 및 UI 버튼 활성화 구현

2차 음정 인식 기능과 시각 효과의 추가 우선

3차 : 악보와의 비교 테스트. 일단 노트 이벤트만 인식

4차 : 추가적 음악 표현의 인식 테스트

중간마다 : 안정성 테스트

6. 참고 가능 요소

Web Audio API의 음정 인식 예제

[Tone.js](#)

<https://musci.io/ko/pitch-detector>